Next Generation Internet of Things (IoT) and Cloud Security Solutions – Research Article

International Journal of Distributed Sensor Networks

International Journal of Distributed Sensor Networks
2018, Vol. 14(9)

DOI: 10.1177/1550147718801535
journals.sagepub.com/home/dsn
SAGE

An analysis on the revoking mechanisms for JSON Web Tokens

László Viktor Jánoky[1], János Levendovszky[2] and Péter Ekler[1]

Abstract

JSON Web Tokens provide a scalable solution with significant performance benefits for user access control in decentralized, large-scale distributed systems. Such examples would entail cloud-based, micro-services styled systems or typical Internet of Things solutions. One of the obstacles still preventing the wide-spread use of JSON Web Token–based access control is the problem of invalidating the issued tokens upon clients leaving the system. Token invalidation presently takes a considerable processing overhead or a drastically increased architectural complexity. Solving this problem without losing the main benefits of JSON Web Tokens still remains an open challenge which will be addressed in the article. We are going to propose some solutions to implement low-complexity token revocations and compare their characteristics in different environments with the traditional solutions. The proposed solutions have the benefit of preserving the advantages of JSON Web Tokens, while also adhering to stronger security constraints and possessing a finely tuneable performance cost.

Keywords

JSON Web Tokens, security, access control, distributed systems

Date received: 30 December 2017; accepted: 22 August 2018

Handling Editor: Vinod Kumar Verma

Introduction

The main goal of JSON Web Tokens (JWT)[1] is to provide an open and secure way for representing claims between two parties. Extending upon this concept, there are multiple implementations using JWT as the basis for user authentication and access control. In this section, we provide a quick overview about the basics of these approaches and outline their main benefits, as well as their disadvantages. Furthermore, we identify those application scenarios where these solutions may prove to be ideal.

In the "Problem statement" section, we delve into the open problems hindering the JWT-based approaches, followed by the brief descriptions of some initial attempts at solving them.

In the "Proposed solution" section, we propose a new solution for the problems outlined previously. We describe the corresponding principles, detail the behavior, and go through some potential issues and their mitigation.

In the "Comparison of methods" section, we compare all the previously described solutions by various metrics based on models and measurements. We outline the benefits and drawbacks of each solution and analyze their impact on the overall system architecture. We draw conclusions as the result of these and show which should be used in different cases.

[1]Department of Automation and Applied Informatics, Budapest University of Technology and Economics, Budapest, Hungary
[2]Department of Telecommunications and Media Informatics, Budapest University of Technology and Economics, Budapest, Hungary

Corresponding author:
László Viktor Jánoky, Department of Automation and Applied Informatics, Budapest University of Technology and Economics, Magyar tudósok krt. 2, Budapest 1117, Hungary.
Email: laszlo.janoky@aut.bme.hu

Finally, in the "Conclusion" section, we conclude the article by reviewing the results, reiterating their significance and potential uses, and finally outlining some future directions.

JWT-based authentication

In a typical JWT-based access control scheme, the initial user authentication is done using a traditional method, for example, providing a username and password combination.

Upon successful authentication, the authorization details of the user, such as identifiers, roles, and permissions, are packed into a JWT. The token is then encrypted using a shared encryption key (called the JWT secret) ensuring verifiability at each secured endpoint.

This token is passed along each subsequent request by the client (known as a "bearer token"), decrypted and unpacked by the service, and the access control data from the token is retrieved. In the security terminology, we usually call these tokens as *access tokens*.

Accessing a secured resource through this scheme entails the following procedure:

1. The client sends an authentication request using a pre-defined method, such as a username–password combination.
2. The authentication service, after checking the credentials of the user, creates a JWT from the retrieved user authorization data. This token is encrypted using the JWT secret and then returned to the client, but not stored in the server.
3. The client passes along this token with the request to a secured resource.
4. The secured service receives the request, decrypts the attached token, and unpacks the user authentication data from it. Using this information, the request can be authorized or denied, and a proper response is sent.

For added user comfort, most solutions integrate another layer of authentication by providing refresh tokens (similar to the concept known from the OAuth protocol).[2] These long-lived tokens are assigned upon successful authentication and can be used to request new access tokens. This can be done invisibly to the user, unlike a request for username and password.

Main benefits

There are two main benefits of a JWT-based authentication scheme, better scaling/performance and decreased complexity when dealing with distributed systems.

The scaling potential comes from the fact that the trusted authentication and authorization information is passed along the communication itself, instead of server-side retrieval. Interpreted from another perspective, this means that the user state data are retrieved from the stable storage upon login and kept in the communication for the duration of the session.

The key feature, allowing the transition of state to the communication, is that the client cannot change the information contained in the token. This is ensured using encryption or digital signing and message authentication codes (MACs) as described in the JSON Web Signature document.[3]

Another advantage is the decreased complexity and decentralization of authorization when dealing with distributed systems. This is can be achieved thanks to the token itself containing all the necessary information, making connections between granting and secured components unnecessary.

Application scenarios

As JWT-based user access control solution has some unique characteristics. It can be ideal in certain cases and less desired in others. It is best used in scenarios where its strong points, namely, performance at scale and decreased complexity, are the defining requirements.

One applicable area would be the field of large-scale distributed systems, such as the world of cloud-based micro-services.[4] In these applications, the core functionality is provided through several decentralized, independent services. Despite the basic principles of this architectural style, some functions remain hard to decentralize, such as access control. In these cases, a JWT-based approach is preferable, if it is capable of fulfilling the security requirements.

Another potential field of application is the emerging world of Internet of Things (IoT) platforms. These solutions face unique challenges in several, previously well-established fields,[5] one such field is security. Security concerns regarding IoT applications are non-trivial from the standpoint of damage a possible attack could cause. An alarming example to this fact would be the largest denial-of-service (DoS) attack, the Dyn 2016 attack, which was carried out with hijacked IoT devices.[6] JWT-based solutions could be beneficial in both decentralization and simplifying architectural complexity mandated by security solutions.

In both environments, scaling is a focal point of the architecture, and centralized state is a major concern with scaling.[7,8] By moving a centralized function partly to the individual components, a JWT-based solution would prove to be beneficial for scaling.

Problem statement

Both the main strengths and problems when using JWT arises from the decentralization of the client authorization state. On one hand, it is easier to access the state in

the communication context when it is distributed to each client; on the other hand, it becomes harder to change from a centralized location.

In this section, we investigate some common scenarios where these problems arise and show some common solutions to them.

Logout problem

One of the main problems with a JWT is that it cannot be easily revoked at will, as token validity is not explicitly stored in the server. This makes invalidation much harder with JWT than in case of a traditional token-based solution.

This can be problematic if a user session has to be invalidated, either because the user has logged out or their access was revoked from a protected resource.

The source of the problem is that the validity of a token is determined by the contents and ability of the server to decrypt and unpack it. This means that distinguishing between a valid and invalid token is only possible based on the contextual information and the data stored in the token itself.

User update problem

Another problem with JWT is that the contents of the token (claims) can become outdated if the data it was constructed from change. Because of that, it is usually not recommended to store frequently changing data in the token, for example, it is unwise to store the balance of a user in the token.

The contents of a JWT, when used for user access control, are sensitive to the update problem. Typically, when changing the permissions of a user, it is desirable for the change to take place immediately. A user should not be able to access a secured resource from which their access was revoked from.

It is possible to create a new token as the user data are updated, but there is no way to force the client to forget the old token and use the new one, thus a malicious user could take advantage of that.

The update problem can be broken down into two parts, creating and delivering the new token to the client and invalidating the older ones. The delivery of the updated token can be done the same way as the delivery of the original token, so the first part of the problem is straightforward. The second part can be reduced to the same problem as the logout problem, which still needs to be solved however.

Presently used solutions

In this section, we describe some previously proposed solutions to the aforementioned problems. We also show the impact of these solutions on the system from both the standpoint of performance and architectural complexity.

Regarding the performance aspect, each solution imposes two kinds of load on the system:

1. *Access load.* The cost associated with accessing a secured resource, denoted with L_{acc}.
2. *Token acquisition load.* The cost associated with acquiring a new access token, denoted with L_{tkn}.

In most cases, we can focus on the L_{tkn} token acquisition load, as access load will be entirely dependent on the number of secured service access, which is a system characteristic and independent of the different solutions. In the formulas where applicable, we will denote the load imposed by checking a single access token for validity as l and the number of secured resource accesses in a unit of time as r. This means that in most cases, the following can be said

$$L_{acc} = l*r$$

Short-lived tokens

A common solution to the log-out problem is to include an expiry date in the token itself and stop providing new tokens to logged out users by invalidating their token acquisition method. This is a straightforward solution, but it has some drawbacks to consider. In case our tokens have a lifetime of $T_{lifetime}$, the following effects would apply to our system.

Token revoking will not be instantaneous; in the worst case, it could take up to $T_{lifetime}$ time for the last token assigned to the client to expire.

Every token will expire periodically, even of those clients we do not want to lock out of the system. Depending on the new token acquisition method, this could provide a significant performance overhead or a drastic drop in user experience (if the user has to repeatedly log in).

Calculating with a constant number of clients N, this method will impose a load of

$$L_{tkn} = N * \frac{1}{T_{lifetime}}$$

which is the token acquisition per second on the system. All these acquisitions will have their respective resource costs associated with them in database operations, network traffic, and computational power.

The solution does not impose further load on the system in the form of access load.

Black listing

Another common approach is to keep a blacklist of revoked tokens and check each token used in the

request against this list. Thanks to modern key-value stores, the checking operation can be done in $O(1)$ time complexity, but it must be done for each token in each request, as we cannot know whether it is blacklisted or not at that time. This introduces a considerable performance overhead for each request.

Blacklisting also brings back the problem of having a shared, centralized place for storing the blacklisted tokens. The lack of a centralized state was one of the main advantages of the JWT approach, using a blacklist defeats this point.

The blacklist should be able to store the maximum necessary number of tokens, and no longer relevant tokens should be removed from it. Both invalidation and cleaning can be done using creation and expiration time information stored in the tokens themselves, like it is done in the Passport solution.[9]

Thus, to provide an upper bound for the size of the blacklist, it is necessary for tokens to have a limited lifetime, otherwise the blacklist could grow to unlimited size. This means the previous

$$L_{tkn} = N * \frac{1}{T_{lifetime}}$$

acquisition load can be also included here, albeit with a much longer lifetime. Moreover, this solution also imposes an additional access load of

$$L_{acc} = (l + b) * r$$

where b denotes the cost of checking a token against the blacklist.

In summary, the blacklist provides a working solution at the cost of a performance overhead, increased storage requirements, and greater centralization and architectural complexity, with all its implications. One could argue that by having to maintain a centralized storage for invalid tokens, it would be just as easy to maintain one for valid tokens, leaving the extra costs such as encryption overhead with JWT behind.

Changing the JWT secret

To achieve immediate token invalidation, another option is to change the JWT secret itself, invalidating every encrypted token using the old secret at the same time. The main advantage of this approach is that it does not require a centralized data storage and invalidation is instantaneous.

When revoking events are scarce, changing the JWT secret might yield a better alternative than short-lived tokens, as new token acquisition events are bound to follow the revoking events, instead of a fixed lifetime period.

The biggest problem with this method is that it cannot distinguish between individual clients and tokens, thus revoking one token equals to revoking all. In cases where there are only a relatively small number of active clients in the system, this overhead can be negligible, but having an increased number of clients in the system, this may change drastically. Hence, the main load in this case comes from token acquisition expressed as

$$L_{tkn} = \frac{1}{T_{rvk}}$$

where T_{rvk} denotes the average time between token revocation events.

To characterize the system load, we must provide a way to determine T_{rvk}. For this, let us assume a worst-case scenario, in which our system operates at maximum capacity and each time a client leaves a new one enters immediately. Also, assuming the worst case, we make token invalidation mandatory when a client leaves (i.e. we have a security critical system). Of course, this is not an ideal use case for a JWT-based scheme, but the following calculations could also be applied to a larger system, with probabilistic log-out chances instead of mandatory.

Let us first assume that clients are indexed with $i = 1, \ldots, N$ and each client has a probability density function $f_i(t), i = 1, \ldots, N$ describing the distribution of their session length in the system. These distributions are specific to the given set of clients and can be obtained by measuring the actual client behavior to approximate these distributions.

In this implementation, a single client ending their session corresponds to global token revocation as changing the JWT secret causes all tokens to be invalidated. Thus, the time between token revocation events can be described by a random variable, denoted by ξ. The probability distribution function of the random variable ξ can be given as

$$F_{\xi}(T) = P(\xi < T) = 1 - \prod_{i=1}^{N} \left(\int_{T}^{\infty} f_i(t)dt \right)$$

Let u denote the minimum time between token revocation events; in this case, the probability of the time interval between two revocations can be calculated as

$$G(u) = P\left(\min_i \xi_i \leqslant u \right)$$

This probability gives us the most likely value for global token revocation (denoted by T_{rvk}) based on the client behavior by setting

$$T_{rvk} = \arg\max_{0<u<\infty} G(u)$$

By using the value of T_{rvk}, the token revocation load L_{tkn} can be calculated as follows

$$L_{tkn} = N * \frac{1}{T_{rvk}}$$

Proposed solution

In this section, we introduce a possible solution to some of the problems described previously. We achieve this by combining different approaches from both the JWT world and from other well-known access control schemes and introducing novelty solutions to overcome their shortcomings.

Architecture

The proposed scheme uses two kind of tokens:

1. *Access token*. A JWT containing the client identifier and authorization data. Optionally, for increased security, it may also contain an expiration date.
2. *Refresh token*. A longer lived, single-use token used to retrieve new tokens by the client, not necessarily JWT.

The model consists of the following architectural elements and archetypes, typical for systems using the JWT authorization scheme.

1. *User authorization and access service (UAA service)*. Service responsible for storing and providing user authorization data, issuing new tokens of both types.
2. *Refresh token store*. A data storage component accessible by the *UAA service*, used to store refresh tokens by the associated client identifiers. Typically, a key-value storage in implementation.
3. *Secured service*. A resource on which access control is required. A client can only access this service if they have a valid token, granting them this privilege. The token is unpacked using the *JWT secret* and access to this service is based on its content.

The *UAA service* and *refresh token store* represent the centralized components of authentication, similar components can be found in any typical JWT-based system. The main advantage of JWT is that these services can be isolated from the rest of the system, a secured service can be accessed without interacting with them. Nevertheless, they do represent a scaling difficulty, but tackling this challenge is not in the scope of any JWT-based solution, neither this article.

The process of accessing a secured resource, retrieving a new access token, and a client logging out is shown in Figure 1.

Minimizing token revocation load

As access tokens are JWT tokens in our solution, we must provide a way for their invalidation. We chose the secret changing method for this task, with an addition to token generation strategy to overcome its previously discussed shortcoming.

A potential issue with this approach is the inherent problem of JWT secret changing, namely, any token revocation means all the tokens are revoked. As shown previously, the number of total revocation events is determined by T_{rvk} (average time between token revocations) which depends on two factors:

1. The client session length probability distribution
2. The number of concurrent clients in the system

In order to minimize L_{tkn}, we have to maximize the value of T_{rvk}.

Client session lengths are given, so the only other factor to influence the system is the number of concurrent clients. To minimize this number, we can partition the population to several groups, where each group uses a different JWT secret. This means that the revocations would only affect clients in the same group.

A possible way to implement this partitioning is to include a group identifier data part in the token, readable by all the services. This allows the service to retrieve the proper JWT secret for the decryption of the token. Initial group assignments can be done randomly, using a hash function, or a custom algorithm based on client behavior.

One possible inspiration for a custom algorithm could come from the field of garbage collection. For example, in generational garbage collection,[10] objects are classified into "generations" based on their current/expected lifetime, in order to make collection easier. The same could be done for clients, by having the more probability to log-out clients assigned to the same group. This way if there are multiple revocation events occurring, one could forego the actual secret change (e.g. client one causes a secret change and client two tries to log-out, but because of client one, they already have their token revoked, therefore they can safely leave the system without further action).

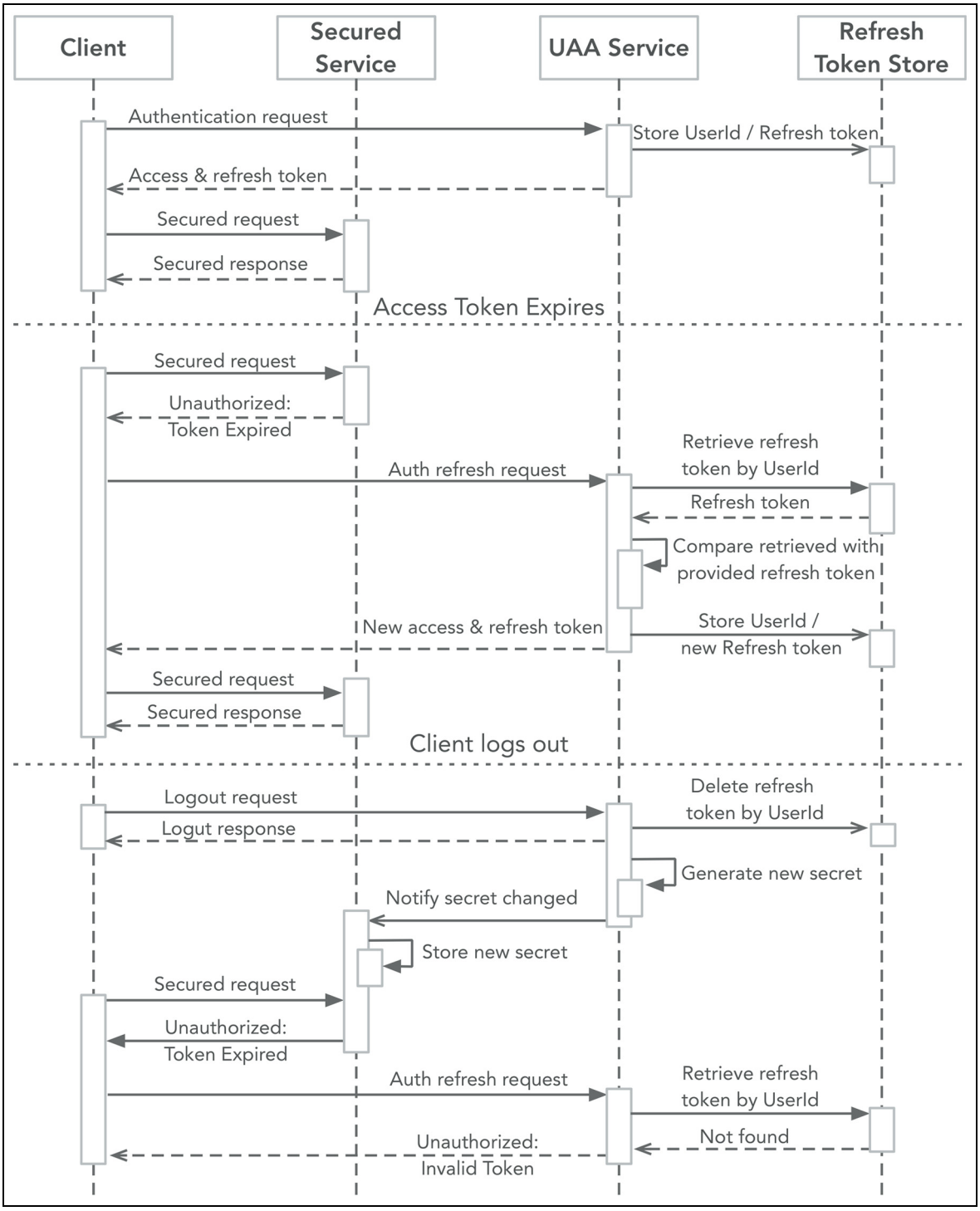

Figure 1. Basic working of the proposed solution.

With the introduction of different client groups, the previously used formula for JWT secret changing still remains valid, but instead of the whole system, it will describe a single group.

To describe the token revocation load on the whole system, we could use the following formula, where k is the number of client groups and T_{rvk} stands for the token revocation time in a group like this

$$L_{tkn} = k\left(\frac{N}{k} * \frac{1}{T_{rvk}}\right) = N * \frac{1}{T_{rvk}}$$

After simplifying, this formula turns out to be the same as for the plain JWT secret, the main difference being T_{rvk}. In case of the plain JWT secret change approach, T_{rvk} is determined by the size of the client population N, while in our case, it depends on the client group size N/k.

Synchronizing JWT secrets across the system

One of the requirements of this approach is to be able to synchronize the *JWT secret* to each *secured service*, anytime it changes. The secret itself is a sensitive information, so secure channels should be employed when transferring it.

If we use the client hashing approach described previously, we have a greater volume of keys to deal with. One possible approach would be to generate the secret using a set synchronized cryptographically secure pseudorandom number generators,[11] creating a kind of rolling code for each group of clients. This would mean that only the key change events have to propagate, not the actual keys themselves, greatly reducing the associated performance cost.

Furthermore, this solution would be more robust to loss of secret change events; when receiving an unintelligible token, the secured service could check the next key if its valid for decryption. If it is, then we can infer that we probably have missed a key change event and react accordingly.

Taking this approach further, it is possible to entirely omit explicit secret synchronization at the cost of increased revocation latency. In this case, each *secured service* relies on the previously described pseudo random number generator method to create a series of *JWT secrets*. When a token is revoked, the associated series is shifted to the next secret at the *UAA service*, and then new tokens are issued with the new key. When a service receives a token signed with the new key, it can decode it by checking it against the upcoming values in the local secret series. If a match is found, the local series of the service is set to the matching point, and every further token signed with the previous keys will be considered invalid.

The advantage of this approach is that apart from the initial setup of the pseudo random generators, no further synchronization is required between the secured services and the centralized infrastructure. The events of secret changes are propagated implicitly trough the system by new tokens using the new secrets.

Mitigating the effects of DoS attacks

The system would be more vulnerable for DoS[12] type of attacks because of the increased controlling challenges associated with its distributed nature[13] and load characteristics of the previously discussed global token revocation.

A relatively few number of clients could cause a huge number of token revocation events. Client grouping would somewhat make these kinds of attacks harder to carry out, nevertheless still possible. A dedicated attacker could fill all the client groups with malicious clients and carry on their attack against the system.

A typical attack exploiting the global revocation would look like a sequence of authentications and log-outs. One possible way to mitigate this threat is to limit the number of token revocations a client can trigger, for example, by adding an exponential timeout for each subsequent authentication request. This way each malicious client could only generate a few token revocations before they would be hit by a longer timeout.

Comparison of methods

This section deals with the comparison of the different JWT invalidation methods. The comparison criteria for each method can be divided into two classes: we have functional and non-functional criteria. Functional criteria contain measurable aspects of a solution, such as the performance impact on the system, while non-functional criteria deals with more abstract terms, such as architectural complexity.

Non-functional overview

The non-functional aspects which we observe are as follows.

Architectural complexity. The complexity associated with a solution includes the number of necessary components and interactions between them, and also the amount of support infrastructure required. Generally speaking, the higher the architectural complexity, the more safeguards have to be put in place to implement a robust system.

Invalidation latency. It is the general time delay between the intent to invalidate a token and the actual point in time when it can no longer be used to access a secured resource.

Acquisition frequency. It is the amount of additional token invalidations (and therefore new token acquisitions for the staying clients) caused by the method in a given time-frame.

Scalability. It is the ability of the system to scale with the number of clients, from the perspective of log-out

Table 1. Comparison of different JWT revoking methods.

Solution	Short lived	Black list	Secret change	Proposed
Architectural complexity	Low	High	Low	Medium
Invalidation latency	Medium	Instant	Instant	Instant
Acquisition frequency	High	N/a	High	Variable
Scalability	Linear	Bad	Very bad	Linear

handling. One of the main factors considering scalability is how the system handles the increased number of revocation events associated with the increase in client numbers (Table 1).

Performance comparison

Different methods have different load profiles depending on different system metrics as shown in the previous sections. In this section, we show how different solutions affect the system from a performance standpoint, with an example based on data measured from a real-world application.

The source of the data is the backing service of a mobile application used for campus-wide information services called the BME-VIK App.[14] We determined the typical session lengths in the system using the log files available, similarly to the method described in section 4.2 by M. Arlitt[15] in their work on characterizing web user sessions. In our analysis, we used a threshold of $t = 3600$ s.

With the raw session data, the next step was to provide a model for predicting client staying times in the system (client session lengths).

To do this, we used an exponential model to approximate the client session lengths. Our approximation provides a 0.1% accuracy, which can be considered good in this case. The original data and the approximation can be seen in Figure 2.

Using this model to simulate client session distribution, we can calculate how the system would react with different number of clients in each case. In this comparison (Figure 3), we can see the L_{tkn} load imposed by the *short-lived tokens* approach, the plain *secret change* method, and our solution (with $T_{lifetime} = 60$, $N/k = 10$ and session lengths according to the measured data).

We chose to omit the blacklist approach from this comparison as the nature of load imposed by that solution is fundamentally different from this three. If a token lifetime limit is used for maintaining the blacklist (as it should be to, avoid growing list sizes), it can be considered a sub-case of short-lifetime tokens.

The load formulas can be given as

$$L_{tkn} = N * \frac{1}{T_{lifetime}}$$

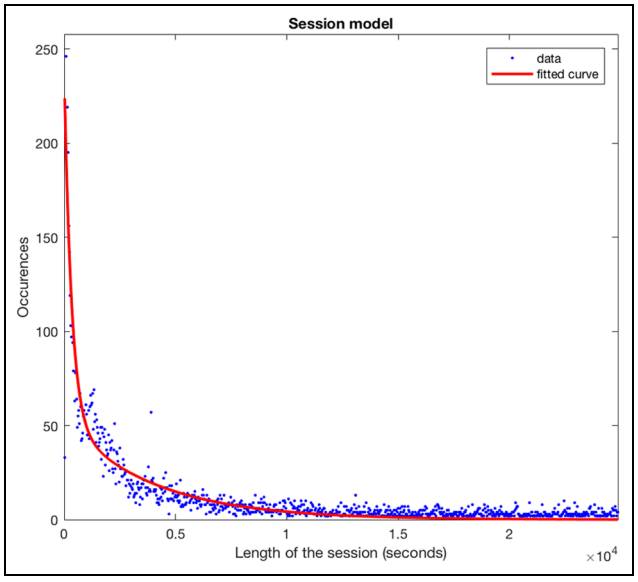

Figure 2. Session length data and approximation.

for short-lived tokens, and as

$$L_{tkn} = N * \frac{1}{T_{rvk}}$$

for both the plain JWT secret change and for our proposed solution.

We can see that the differentiating factor in these formulas are the token lifetime variable:

1. In case of *short-lived tokens*, it is a constant value, therefore the load function is a linear function.
2. In the plain secret change method, T_{rvk} is the function of the client population size N, causing the load function to be also dependent on it. In our measurements, it appears that this relationship is not linear, but exponential.
3. In our solution, T_{rvk} is a function of the constant k, therefore it can also be considered a constant, making our load function linear.

Based on this, we can summarize the performance comparison. We showed that, plain JWT secret change methods are unsuitable for large-scale use due to their

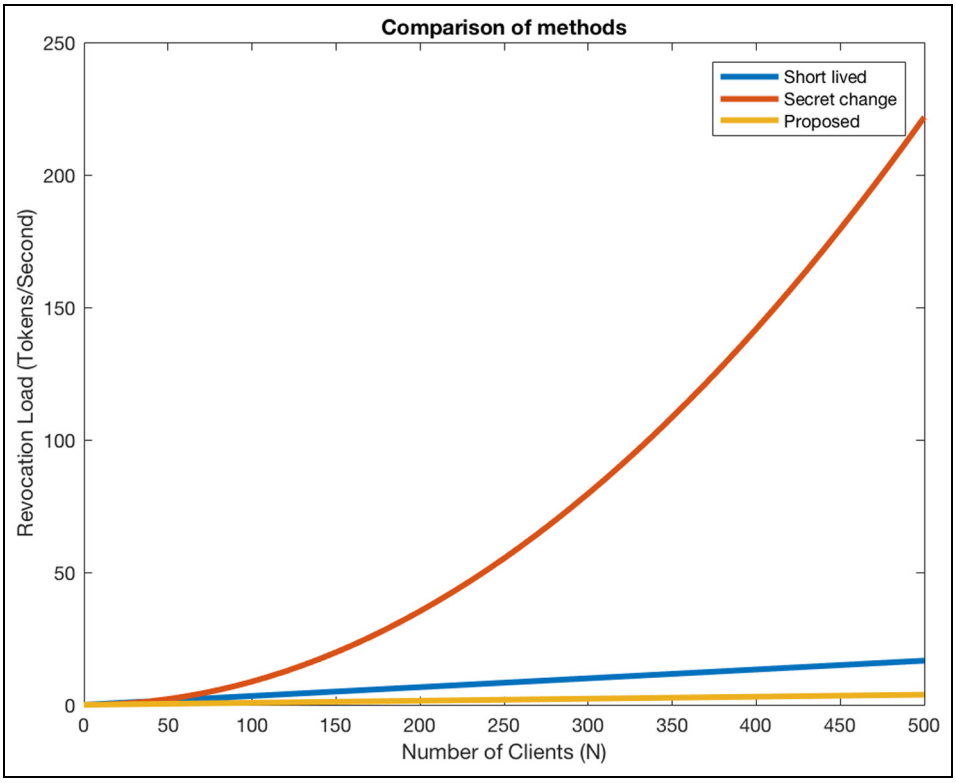

Figure 3. Example token load functions of different methods.

non-linear load functions. It can also be seen that our proposed solution has the same class of system load functions as the industry standard, short-lifetime tokens (with additional flexibility to go even lower in certain cases, seen in our example).

It is also worth noting that we are looking at a worst-case scenario from the perspective of our solution, a more realistic usage would lead to much reduced system load. The same cannot be said from the short-lifetime approach, in that case load would be the same in both scenarios.

Conclusion

In our work, we set out to analyze and improve on the JWT access control solutions, as they could play a key role in the field of security and access control, especially in large-scale distributed environments, such as IoT and cloud systems.

After introducing the basic principles of the field, we showed some still open problems hindering the adaptation of the approach. For these problems, we examined some already proposed solutions and analyzed their impact on the system.

In the next part, we proposed a solution and detailed its workings. We highlighted the key focus points of the solution and challenges it should overcome.

In the "Comparison of methods" section, we compared the previously proposed solutions and our solution from different perspectives. First, we enumerated the non-functional comparison points, such as architectural complexity and scalability. Next, we examined performance metrics, defined the load characteristics of each solution, and showed their effect on a real system, based on measured data.

We concluded that our solution provides instantaneous token revocation, therefore the strictest possible security level in the field. We also proved that even in the worst case, our solution keeps the linear system load characteristics of the industry standard *short-lifetime tokens* approach. While doing this, our solution also retains the main advantages of JWT-based solutions, namely, decentralization and scalability. The only drawback is the increased architectural complexity necessary to propagate secret change events, but for even that we showed some ways to mitigate its effects.

One possible direction to investigate would be a way to improve the effectiveness of this solution by introducing special grouping algorithms for client groups. The goal with these algorithms would be to maximize time between token revocation events. We believe that studying client behavior based on known client characteristics could lead to the creation of a behavioral model, which could be used for this optimization purpose in the future.

Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: The research reported in this paper was supported by the BME-Artificial Intelligence FIKP grant of EMMI (BME FIKP-MI/SC) and by the National Research, Development, and Innovation Fund of Hungary in the frame of FIEK_16-1-2016-0007 (Higher Education and Industrial Cooperation Center) project.

ORCID iD

László Viktor Jánoky https://orcid.org/0000-0002-9070-9026

References

1. Jones M, Bradley J and Sakimura N. JSON Web Token (JWT). RFC 7519, RFC Editor 2015, http://www.rfc-editor.org/rfc/rfc7519.txt
2. Hardt D. The OAuth 2.0 authorization framework. RFC 6749, RFC Editor 2012, http://www.rfc-editor.org/rfc/rfc6749.txt
3. Jones M, Bradley J and Sakimura N. JSON Web Signature (JWS). RFC 7515, RFC Editor 2015, http://www.rfc-editor.org/rfc/rfc7515.txt
4. Namiot D and Sneps-Sneppe M. On micro-services architecture. *Int J Open Inf Tech* 2014; 2(9): 24–27.
5. Stankovic JA. Research directions for the internet of things. *IEEE Internet Things* 2014; 1(1): 3–9.
6. Woolf N. DDoS attack that disrupted internet was largest of its kind in history, experts say. *The Guardian*, 2016, https://www.theguardian.com/technology/2016/oct/26/ddos-attack-dyn-mirai-botnet
7. Tanenbaum AS and van Steen M. *Distributed Systems: principles and paradigms*. Upper Saddle River, NJ: Prentice Hall, 2007.
8. Babaoğlu O and Marzullo K. Consistent global states of distributed systems: fundamental concepts and mechanisms. In: Mullender (ed.) *Distributed systems*. New York: ACM Press/Addison-Wesley Publishing Co., 1993, pp.53–96.
9. dWTV. Learn how to revoke JSON Web Tokens, 2017, tps://developer.ibm.com/tv/learn-how-to-revoke-json-web-tokens/
10. Wilson PR. Uniprocessor garbage collection techniques. In: *Proceedings of the international workshop on memory management*, Malo, 17–19 September 1992, pp. 1–42. Basel: Springer.
11. Blum M and Micali S. How to generate cryptographically strong sequences of pseudorandom bits. *SIAM J Comput* 1984; 13(4): 850–864.
12. Mirkovic J, Dietrich S, Dittrich D, et al. *Internet denial of service: attack and defense mechanisms (Radia Perlman Computer Networking and Security)*. Upper Saddle River, NJ: Prentice Hall, 2004.
13. Wood AD and Stankovic JA. Denial of service in sensor networks. *Comput* 2002; 35(10): 54–62.
14. BME-AUT. BME-VIK, 2017, https://itunes.apple.com/hu/app/bme-vik/id1148661980
15. Arlitt M. Characterizing web user sessions. *ACM Sigmetrics Perform Eval Rev* 2000; 28(2): 50–63.